



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Edgar Alejandro Letona Mazariegos. **Carné:** 1304424

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Una interfaz en la cuales conectan múltiples aplicaciones que se conectan a través de una base de datos y comparten información

2. ¿Qué es un endpoint?

Una solicitud o una URL la cual se envía o se utiliza para acceder a una API o recursos dentro de la aplicación

3. ¿Qué diferencia existe entre una URL y un endpoint?

Una URL es un enlace que envía o redirige al usuario a un enlace el cual puede visitar, un endpoint es un enlace que manda a llamar información de una API la cual normalmente tiene autenticación

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

Get sirve para consultar o leer datos

Post Sirve para crear o actualizar un nuevo recurso

Put/Patch Actualiza los datos existentes

Delete Elimina un recurso asignado

5. ¿Qué es JSON y por qué es común en las APIs?

Una estructura la cual es más eficiente para trabajar entre aplicaciones, facilitando la transferencia de datos

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un código normalmente de 3 dígitos que se envía como respuesta normalmente a una petición del servidor

200 significa que la llamada fue exitosa

404 es que no existe el parámetro

500 Un error de servidor

7. ¿Qué información se puede encontrar en la respuesta de una API?

Códigos de estado, encabezados y body. Son resultados enviados a través de una solicitud realizada a la API

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

La principal ventaja es que se puede verificar la estructura y arquitectura de la lógica del servidor sin gastar recursos internos de la API antes de ser ejecutados. Reduciendo los errores que pueden darse durante la ejecución real

9. ¿Qué es y para qué sirve Swagger UI?

Es una forma de expresar todo el código en formato JSON para que sea más fácil de interpretar y sea funcional para los usuarios



10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Facilita el uso de los get, post, put/patch y la creación de documentación de estas a partir de una interfaz que muestra información importante para poder gestionar las APIs, endpoints y response de estas mismas

11. Que es un payload y que es un response?

Un payload es una forma de mandar una solicitud dentro del API como un post, un response es un get dentro del API el cual funciona para traer recursos solicitados



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
GET /pet/{petId}

- Indique el método HTTP utilizado.

Método GET

- Indique qué parámetro necesita la solicitud.

petId

- Ejecute la solicitud con Try it out y registre el código HTTP recibido.

The screenshot shows the Swagger UI interface for the Petstore 3 API. The URL bar displays `petstore3.swagger.io/#/pet/getPetById`. The 'Try it out' section shows a cURL command: `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/1' \ -H 'accept: application/json'`. The 'Request URL' is `https://petstore3.swagger.io/api/v3/pet/1`. The 'Server response' section shows a 404 status code with the message 'Error: response status is 404' and a response body of 'Pet not found'. The 'Response headers' section lists various headers including 'access-control-allow-headers', 'access-control-allow-methods', 'access-control-allow-origin', 'access-control-expose-headers', 'content-length', 'content-type', 'date', and 'server'. The 'Responses' table shows a 200 status code for a 'successful operation'. On the right, the DevTools network tab is open, showing a list of requests including 'favicon.ico', 'data:image/svg+xml;...', 'pet', and 'data:image/svg+xml;...'. The 'pet' request is highlighted, showing a 404 status code and a response size of 0.3 kB.

- Explique con sus palabras qué información contiene la respuesta.
Contiene información de la mascota en formato JSON incluyendo datos como ID, nombre, categoría, entre otros.
Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Parameters

No parameters

Request body **required** application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 13044,
  "name": "Skaya",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "https://petstore3.swagger.io/api/v3/img"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute Clear

Get ID

petId **required** ID of pet to return
integer(int64)
(path)
13044

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/13044' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/13044

Server response

Code Details

200

Response body

```
{
  "id": 13044,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "https://petstore3.swagger.io/api/v3/img"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
Método PUT para actualizar la información de un recurso existente
- ¿Qué datos fueron enviados en el body?
Datos incluidos dentro del formato JSON como ID, Nombre, Categoría, Fotografías, etiquetas, y estado
- ¿Qué código HTTP devolvió la API?
Estado: 200 ok apreciado en la captura
- ¿Cómo comprobó que el cambio realmente ocurrió?
Se registro dentro de la consola y también se volvió a comprobar en el método get

The screenshot displays the Swagger UI interface for the petstore3 API. The top section shows the 'Parameters' for the PUT /pet endpoint, with fields for 'petid' (ID of pet that needs to be updated), 'name' (Name of pet that needs to be updated), and 'status' (Status of pet that needs to be updated). The 'petid' field is set to 13044, 'name' is 'Skyra actualizado', and 'status' is 'available'. The 'Execute' button is visible.

Below the parameters, the 'Responses' section shows the 'Curl' command for the PUT request:

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet/13044?name=Skyra&actualizado&status=available' \
  -H 'accept: application/json' \
  -d ''
```

The 'Server response' section shows the 'Code' as 200 and the 'Response body' as a JSON object:

```
{
  "id": 13044,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Skyra actualizado",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

The 'Response headers' section shows the following headers:

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 151
```

The bottom part of the screenshot shows the DevTools Network tab, displaying a list of requests. The request for the PUT /pet endpoint is highlighted, showing a status of 200 and a size of 0.4 kB.



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
Se utilizó el status
- Explique dónde se envía ese parámetro.
Un parámetro como un query para determinar cuales son los datos relacionados con “available”
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
La API devolverá únicamente los valores de los datos con “pending” en vez de “available”

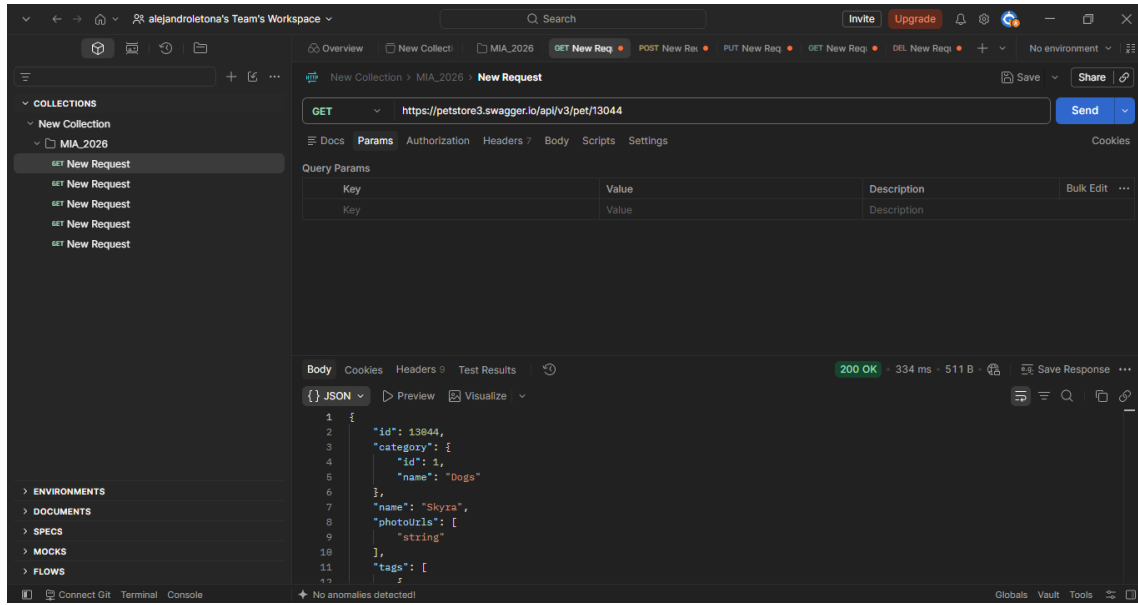
The screenshot displays the Swagger UI interface for the petstore3.swagger.io API. The 'Parameters' section shows a query parameter named 'status' with a description 'Status values that need to be considered for filter'. The value 'available' is selected in the dropdown menu. The 'Execute' button is visible. Below the parameters, the 'Responses' section shows the curl command and the request URL: `curl -X 'GET' \ 'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \ -H 'accept: application/json'` and `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available`. The server response is shown as 200.

On the right, the Chrome DevTools Network tab is open, showing a list of requests. The selected request is a GET request to `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available` with a status of 200. The response size is 14.9 kB and the time taken is 467 ms. The console shows the response details.

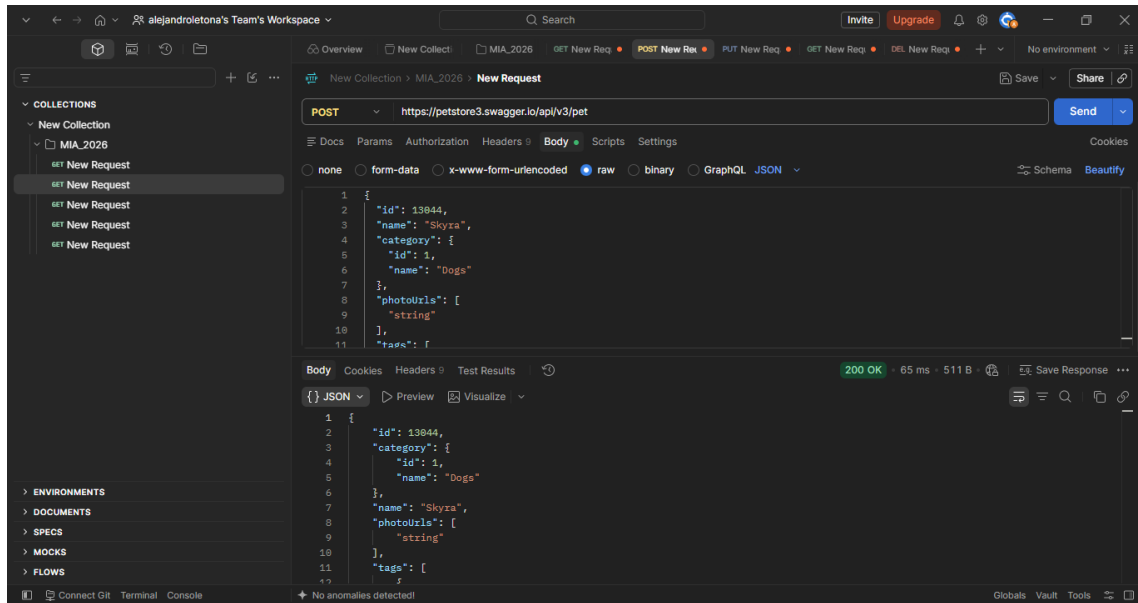


6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota



- PUT – Modificar mascota





- GET – Buscar por status

The screenshot shows a Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available`. The response is a 200 OK status with a JSON body. The JSON body contains a list of pets with the following structure:

id	category	name	photoUrls	tags	status
4	1	Dog 1	0 url1	0 1	available
	name	Dogs	1 url2	1 2	
7	4	Lion 1	0 url1	0 1	available
	name	Lions	1 url2	1 2	

- DELETE – Eliminar mascota

The screenshot shows a Postman interface with a DELETE request to `https://petstore3.swagger.io/api/v3/pet/13044`. The response is a 200 OK status with a JSON body containing the text `Petdeleted`.

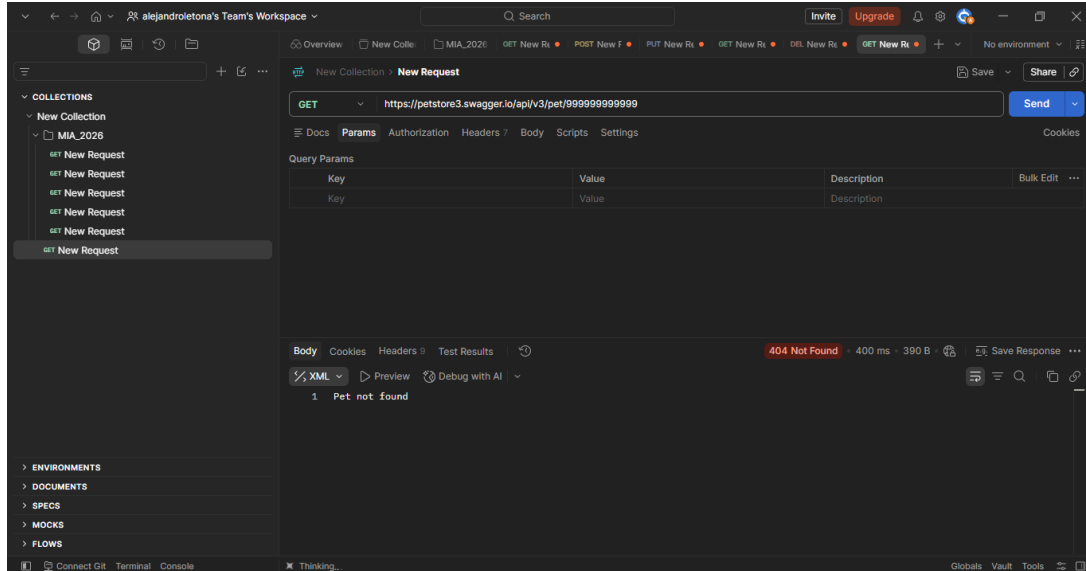
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.



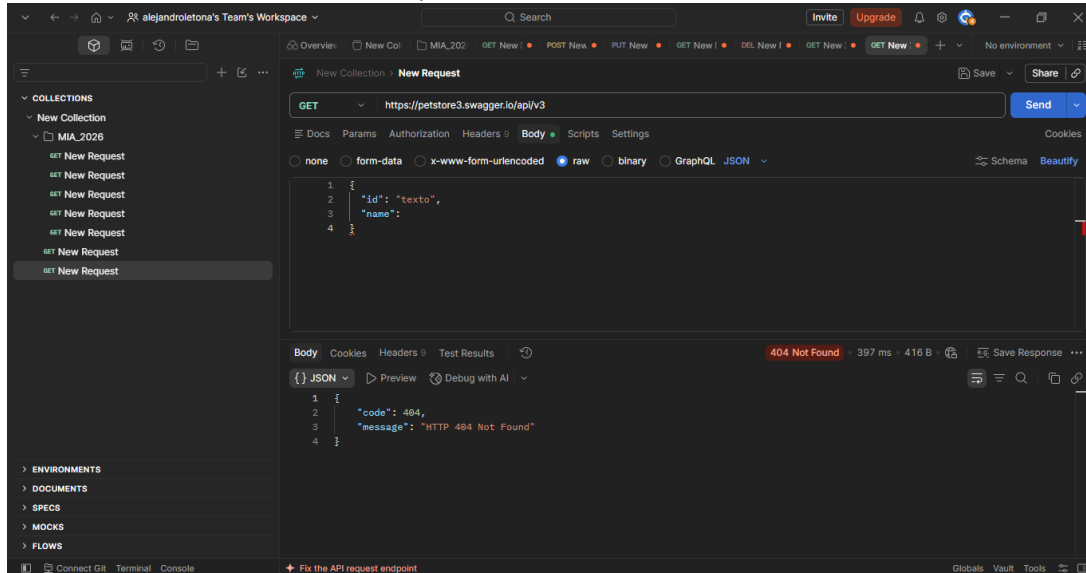
Solicitud: consulta de un ID inexistente.

Esperaba: que la API indicara que la mascota no existe.

Causa: no existe un recurso asociado con ese ID.

El código es 404 el cual significa que no existe

2. Enviar un JSON incorrecto o incompleto.

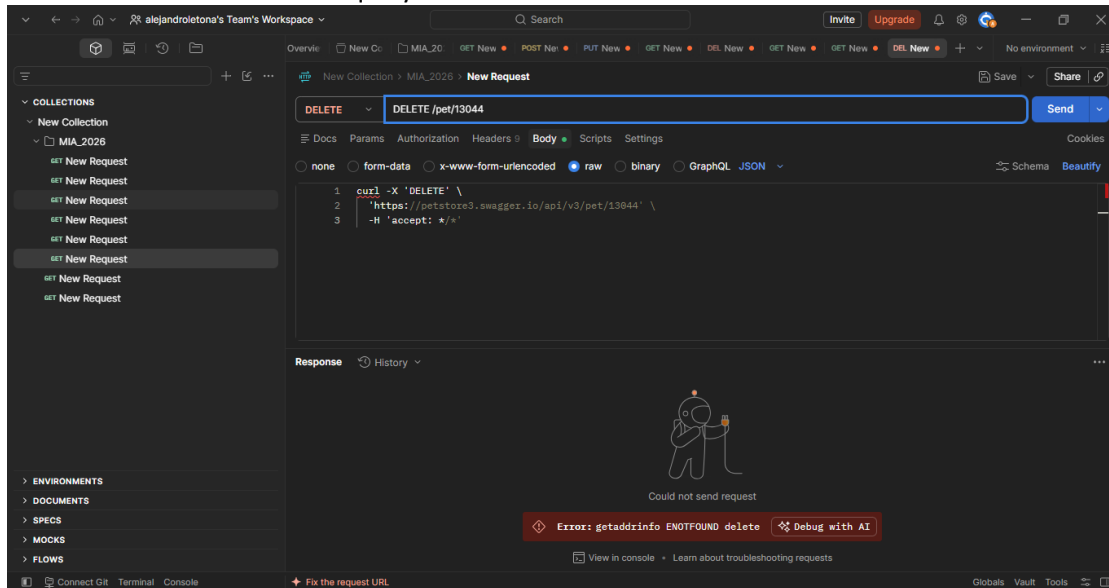


Se esperaba: que la API rechazara la solicitud debido a que el JSON no tiene una estructura válida.

Causa: el cuerpo enviado no cumple con el formato JSON esperado por el endpoint.



3. Intentar eliminar un recurso que ya no existe.



Se esperaba que la API indicara que el recurso ya no existe.

Qué ocurrió realmente: coloca la respuesta exacta obtenida.

Causa: la mascota fue eliminada durante la primera solicitud DELETE, por lo que ya no existe para una segunda eliminación.

Para cada prueba registre:

- Solicitud realizada.
- Código HTTP recibido.
- Respuesta de la API.
- Qué esperaba que ocurriera.
- Qué ocurrió realmente.
- Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.